Overview: Verifier-Guided Agentic OpenROAD Code Generation with Trajectory-Aware Control

I. Introduction

Recent advances in large language models (LLMs) and autonomous agents are transforming the automation of complex engineering workflows. Across many domains, research is moving beyond static or narrowly scripted pipelines toward *agentic frameworks* that can plan, verify, repair, and adapt with limited human intervention. This transition is particularly important in Electronic Design Automation (EDA), where workflows are highly sequential, tightly coupled, and sensitive to errors introduced at intermediate stages. Even small mistakes in scripts, parameters, or tool usage can propagate through the design flow, causing failed runs, degraded quality of results, and substantial manual debugging effort. At the same time, open infrastructures such as *OpenROAD* provide a transparent and reproducible platform for studying these challenges, accelerating research at the intersection of LLMs, agents, and chip design.

Within this broader trend, an important emerging direction is the use of natural language interfaces for chip design, including natural-language-to-physical-design scripting. Prior work has demonstrated the promise of LLM-assisted interaction with OpenROAD through systems such as ChatEDA, OpenROAD Agent, and ORFS-Agent. However, a central bottleneck remains: many existing approaches rely on *expensive tool-in-the-loop correction*. In these systems, the model generates a script, executes it in the real tool, inspects the resulting logs, and then revises the script in the next iteration. Although effective, this process introduces significant latency and cost because every repair cycle depends on a full OpenROAD execution. The problem becomes more severe when script generation is only one component of a larger multi-step design objective.

This paper addresses that bottleneck through a **causal-graph-guided agentic framework** that shifts much of the debugging burden from the external EDA tool to the agent itself. We argue that instability in current EDA agents arises from the lack of structural grounding in the OpenDB design hierarchy. Instead of treating script generation as a flat text-completion problem, we formulate it as a **hierarchical causal graph construction and verification** problem. Our key insight is that every valid EDA program must respect the underlying dependency structure of database objects and operations. For example, object access often follows valid hierarchical paths such as

$$dbDatabase \rightarrow dbChip \rightarrow dbBlock \rightarrow dbInst,$$

while many actions require reasoning over multiple converging paths in a larger causal graph. By equipping the agent with an internal verifier that reasons over this structure before making costly tool calls, we enable a more efficient generation–verification–repair loop.

This structural view is especially important for **natural-language-to-GDS** tasks, which are inherently *long-horizon*. Much like embodied agents in robotics, an EDA agent must decompose a complex goal into smaller subgoals and achieve them step by step. A single high-level prompt may require multiple dependent operations, intermediate checks, and coordinated edits across the design database. Causal graph construction makes this decomposition possible: the agent first identifies the relevant objects, dependencies, and action nodes implied by the prompt, then plans execution as a sequence of graph-grounded subgoals. In this way, causal reasoning provides both structural guidance for code generation and a planning substrate for long-horizon control. In summary, this paper makes the following primary contributions:

- **Hierarchical causal graph construction and retrieval**: We introduce a mechanism that decomposes natural language prompts into formal causal graphs over OpenDB classes and operations. This approach uses **node-specific retrieval-augmented generation (RAG)** to fetch the exact API signatures relevant to each graph node, reducing hallucinations of non-existent methods and ensuring that generation remains aligned with the physical design ontology.
- **Type-aware causal verification**: We propose a staged verifier that performs **symbolic trace analysis** over generated programs. Beyond syntax checking, the verifier applies **trajectory-aware control** to identify *causal gaps*, where the agent skips required hierarchical dependencies, and *type mismatches*, such as treating specialized OpenDB objects (for example, `odb.Point`) as generic Python primitives.
- **Causal error localization and repair**: By aligning generated code with the target causal graph, our framework supports precise fault localization. This allows the controller to preserve correct substeps while patching only the failing graph edges or action nodes, making repair more tractable for complex long-horizon tasks.
- **A progressive experimental pipeline for long-horizon natural-language-to-action generation**: We design an evaluation pipeline to study how causal-graph-based code generation affects long-horizon tasks such as natural-language-to-OpenROAD scripting. To do this, we compare four generation strategies. The first solves each subtask independently in sequence, generating and executing code before moving to the next step. The second also proceeds

Fig. 1. Causal graphs for the Design prompts

sequentially, but incrementally extends a shared causal graph across subtasks. The third progressively expands the causal graph over subtasks and generates the complete code only at the end. The fourth builds the full causal graph from the entire prompt upfront and performs single-pass code generation. This comparison helps reveal the trade-off between stepwise execution feedback and full-task planning.

By positioning causal structure as the governing principle of the agentic loop, our work moves EDA automation toward a more practical and efficient direction. We show that while stronger verification improves performance on short tasks, causal grounding is the key factor that makes autonomous assistance feasible for long-horizon design objectives.

II. Methodology

A. Problem Formulation

We study the problem of generating executable OpenROAD Python scripts directly from natural language design prompts, while reducing reliance on expensive tool-in-the-loop self-correction. Given a task prompt x, the objective is to produce a program y that is not only syntactically valid, but also structurally consistent with the OpenDB hierarchy, compatible with the OpenROAD Python API, and executable in a live OpenROAD session. Unlike short single-step prompts, long-horizon prompts often require multiple dependent subgoals, where later operations rely on objects, state, or design modifications created by earlier steps. This makes flat prompt-to-code generation unreliable, since the model must implicitly reason about both hierarchical object dependencies and multi-step execution order.

To address this, we formulate natural-language-to-OpenROAD generation as a *causal-graph-guided program synthesis* problem. Instead of directly mapping a prompt into code, the system first derives the structural dependencies implied by the task, retrieves node-specific API knowledge, generates code grounded in that structure, and verifies the result before invoking expensive tool execution. When internal verification fails, the system enters a causally grounded repair loop that uses structured feedback to revise only the problematic parts of the candidate program. For long-horizon tasks, this process is applied progressively across sub-prompts while preserving the live OpenROAD session state.

B. Framework Overview

Our framework consists of two levels. The first is a *single-step causal generation pipeline*, which converts one natural language task into verified OpenROAD code. The second is a *sequential long-horizon evaluation pipeline*, which extends the same mechanism to multi-step prompts by solving a sequence of sub-prompts in a shared OpenROAD session.

At the single-step level, the pipeline contains four bootstrap stages: causal graph extraction, node-specific retrieval, graph-guided code generation, and staged verification. If the generated code passes internal verification, it is committed and then executed in OpenROAD, where the tool output serves as the final ground-truth evaluation of correctness. If verification fails, a controller-guided repair loop revises the candidate using structured feedback. At the long-horizon level, each sub-prompt is processed by the same causal agent, but only after all previous sub-prompts have passed. Accepted code from earlier steps is provided as context to later steps, and all committed code executes in the same OpenROAD process so that design state carries forward naturally.

C. Causal Graph Construction from Natural Language

The first stage of the framework converts the natural language task into a structural representation over OpenROAD and OpenDB objects. Rather than treating the prompt as an unconstrained text-generation problem, the system identifies the terminal objects needed to complete the task and the valid acquisition paths required to reach them from the pre-existing `design` object. This stage is implemented with an LLM that is prompted to produce one or more object-acquisition paths together with a final action node.

As illustrated in Fig. 7, each path begins at `openroad.Design` and ends at a task-relevant terminal object such as `odb.dbInst`, `odb.dbNet`, or `odb.dbBTerm`. For action-oriented tasks, the extracted representation may contain multiple independent paths that converge at an action node. For example, a port-to-net connection task requires one path to acquire the boundary terminal and another path to acquire the target net. This multi-path view is important because many OpenROAD operations do not follow a single linear hierarchy; instead, they require several independently acquired objects to be joined by a downstream action.

D. From Linear Chains to Multi-Path Causal Graphs

A key design choice in our framework is moving from linear causal extraction to multi-path causal graph extraction. Existing causal code generation systems often model object acquisition

as a single monotonic chain. In the OpenROAD setting, such a representation may be written as

$$\mathcal{P} = \{\texttt{Design} \rightarrow \texttt{dbBlock} \rightarrow \texttt{dbBTerm} \rightarrow \texttt{dbNet}\}. \quad (1)$$

This representation implicitly assumes that each node must be derived from its immediate predecessor. While this is sufficient for simple read-only traversal tasks, it becomes problematic for action-oriented operations.

Consider the task *"Connect port A to net B"*. A linear representation may incorrectly suggest that the net should be obtained from the boundary terminal, for example through a step such as `bterm.getNet()`. This introduces a *phantom dependency* between `dbBTerm` and `dbNet` that does not reflect the actual OpenROAD API structure. As a result, the generator or verifier may enforce a semantically invalid access path and fail even when the intended operation is conceptually correct.

To address this limitation, we distinguish between two classes of tasks. *Traversal tasks* follow a hierarchical "has-a" relationship and can be represented as linear paths. In contrast, *co-reference action tasks* require two or more independently acquired objects that are later consumed by a common operation such as `connect`, `swap`, or `place`. These tasks are better represented as multi-path causal graphs with converging branches and an explicit action node.

For such tasks, the extractor builds a dependency set rather than a single chain:

$$\begin{aligned} \text{Path 1 (Port):} \quad & \texttt{Design} \rightarrow \texttt{dbBlock} \rightarrow \texttt{dbBTerm} \\ \text{Path 2 (Net):} \quad & \texttt{Design} \rightarrow \texttt{dbBlock} \rightarrow \texttt{dbNet} \\ \text{Action Node:} \quad & \texttt{connect}(\text{Port}, \text{Net}). \end{aligned}$$

Under this formulation, the agent state is satisfied not by a single rigid sequence of calls, but by the availability of the required object types in the current environment. By separating the acquisition of the port and the net, the framework avoids introducing artificial dependencies between unrelated branches.

This graph-based representation also improves robustness during verification. In particular, it prevents the verifier from penalizing valid code for omitting hallucinated intermediate steps, such as forcing `dbNet` to be retrieved through `dbBTerm` when it can instead be acquired directly from `dbBlock`. In implementation, the extracted structure is stored as a set of paths together with a unified edge set. For backward compatibility, one path may still be treated as the primary chain, but retrieval and verification operate on the complete edge set.

E. Node-Specific Retrieval

After graph extraction, the framework performs retrieval over the OpenROAD API database to ground each causal edge with concrete method-level knowledge. Instead of retrieving a single global context for the full prompt, retrieval is performed independently for each edge (s, t) in the graph. The retrieval stage first attempts a strict symbolic match based on return type and function name, and falls back to semantic embedding similarity only when exact matches are unavailable.

This node-specific retrieval design serves two purposes. First, it reduces irrelevant context by restricting the retrieved information to the local transition currently being modeled. Second, it makes hallucination detection easier during verification, since each edge can later be checked against a specific retrieved API expectation. In implementation, the system reads API entries from `RAGAPIs.csv`, embeds their descriptions, and stores the top retrieved method for each edge. The resulting edge-to-method assignments are summarized and passed forward to generation and verification.

F. Graph-Guided Code Generation

Once the causal graph and edge-specific API suggestions are available, the framework generates OpenROAD Python code conditioned on that structure. The generation stage does not ask the LLM to freely synthesize the entire program from the original prompt alone. Instead, the model is given the task, the extracted acquisition paths, the retrieved methods for each edge, and the requirement that objects must be acquired in the correct structural order before implementing the task logic.

In the current implementation, the generator constructs an acquisition context that enumerates the paths, the intermediate object types, and the retrieved methods associated with each edge. The LLM is then instructed to produce flat procedural Python code that runs inside the OpenROAD interactive shell, uses pre-available objects such as `design` and `tech`, and fully implements the task after the required acquisition steps. The output includes both the generated code and an optional diagnosis field, which allows the model to explicitly explain deviations from the retrieved API suggestions when it believes an alternative method is technically more appropriate.

This generation design is intentionally structured. The causal graph constrains *what must be acquired*, the retrieval module constrains *how each transition should be realized*, and the generator is responsible for combining these constraints into executable code.

G. Type-Aware Verification

A key goal of the framework is to reduce reliance on repeated OpenROAD executions for debugging. To achieve this, generated code is verified internally before any expensive tool call. Our implementation uses a staged verifier composed of three components: a static verifier, a causal verifier, and an LLM-based semantic verifier.

The static verifier performs syntax and basic API-level checks. It ensures that the generated code is parsable, non-empty, and structurally valid as a Python program. It also provides broader rule-based checks over known OpenROAD APIs. The causal verifier then evaluates whether the generated code is consistent with the extracted graph. In particular, it checks whether each expected acquisition step is present, whether required methods are called on the correct receiver type, and whether the code respects the intended hierarchy of object dependencies. It also performs API-diff analysis by comparing the method actually used in code for each edge against the method retrieved during the node-specific RAG stage. This makes it possible to flag missing steps, hierarchy violations,

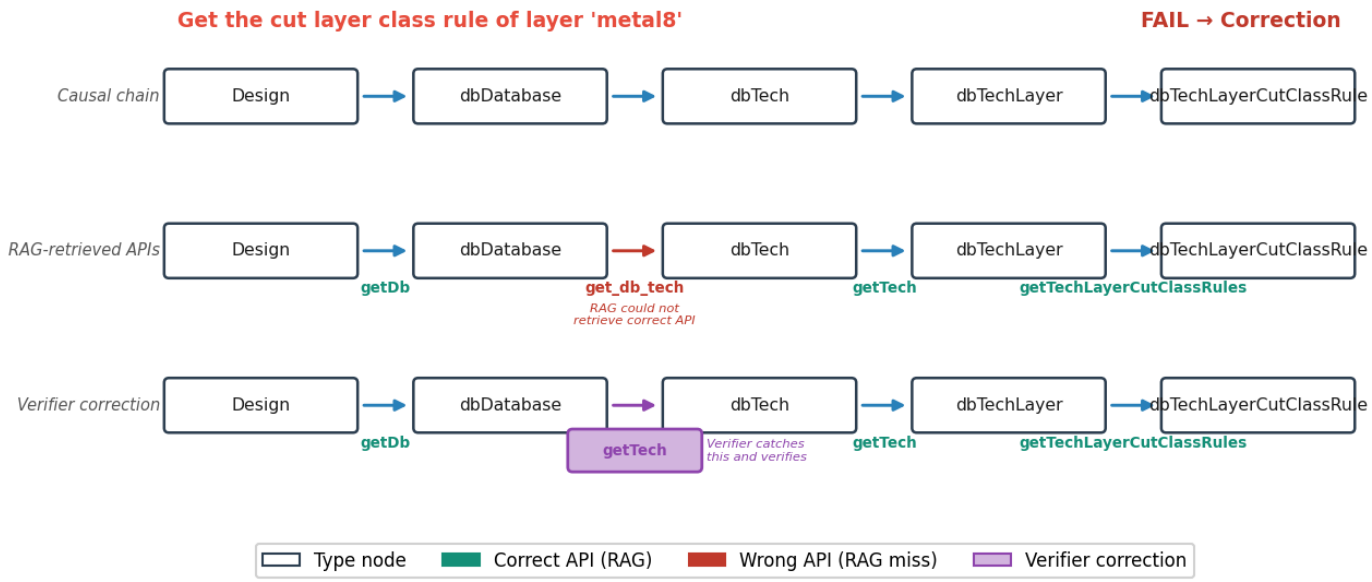

Fig. 2. Verifier detects and repairs API mismatches/hallucinations

and hallucinated method usage at the level of individual graph edges.

Finally, an LLM-based semantic verifier is used as a higher-level check when the earlier stages pass. This semantic stage evaluates whether the generated code is logically consistent with the task and whether it appears executable beyond surface-level syntax and local API correctness. In the current pipeline, code is committed directly only if both the internal structured checks and the semantic verifier accept it; otherwise, the case enters a controller-guided repair loop.

H. Controller-Guided Repair

When the generated code does not pass verification, the framework enters an iterative repair phase. This phase is implemented through a controller-dispatcher design in `run_causal_agent.py`. The controller reads the current agent state, including the task, extracted graph, retrieved APIs, generated code, verifier results, and prior observations, and then decides the next repair action under a bounded action budget.

The repair mechanism is designed to be localized rather than fully regenerative whenever possible. Because the causal verifier returns structured edge-level differences, the controller can focus on specific missing or mismatched transitions instead of regenerating the entire program blindly. The dispatcher then executes the selected action, such as re-generation with additional feedback, targeted re-retrieval, or other repair operations. The history of controller decisions is stored in the agent state together with lessons extracted across attempts. If the budget is exhausted, the best available verified candidate is committed.

Although this controller loop remains simpler than a fully unconstrained autonomous planning agent, it is already more structured than conventional self-correction pipelines because it reasons over explicit graph state, localized verifier outputs, and bounded repair actions.

I. Sequential Extension for Long-Horizon Tasks

To support long-horizon natural-language-to-action generation, we extend the single-task causal pipeline to a sequential multi-step setting. This is implemented in `run_causal_agent_sequential.py`, which reads each case from an action database containing a complex prompt and up to three progressively harder sub-prompts. The user can control how many levels are evaluated through the `--level` argument.

For each case, the system opens a fresh OpenROAD process and keeps that process alive across all sub-prompts in the case. The first sub-prompt is passed to the causal agent pipeline described above. If that step produces committed code and the code executes successfully in OpenROAD, the code is treated as accepted and the system proceeds to the next sub-prompt. If a step fails in OpenROAD, all later steps are skipped for that case. This design ensures that long-horizon evaluation respects true execution dependencies rather than assuming that later steps can succeed independently of earlier state.

An important feature of the sequential setting is that all previously accepted code is provided as context to the next step. However, that context is marked as already executed and must not be re-run. This means the agent for step k sees not only the current sub-prompt, but also the design state established by steps $1, \ldots, k-1$. In this way, long-horizon generation becomes a stateful incremental synthesis problem rather than a collection of isolated short prompts.

J. Progressive Long-Horizon Generation Strategy

The sequential implementation corresponds to one specific long-horizon strategy: *sequential subtask execution with intermediate validation*. For each sub-prompt, the system constructs a local causal graph, retrieves edge-specific APIs, generates code, verifies it internally, and then executes it in the same live

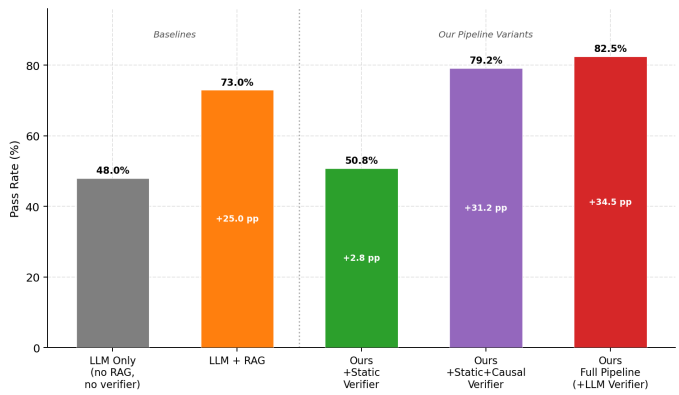

Fig. 3.

III. Results

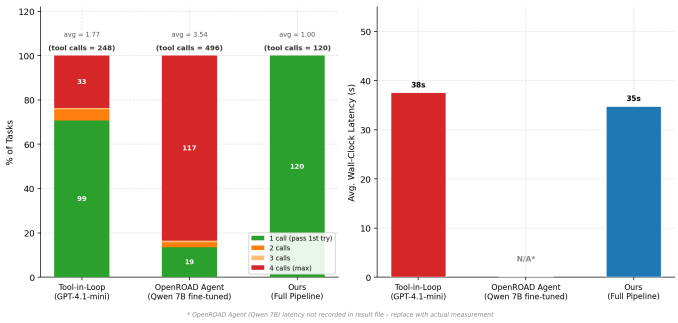

Fig. 4.

OpenROAD session. Only after a subtask passes is the next subtask attempted.

This design has two advantages. First, it allows failures to be localized early, which prevents error propagation across later steps. Second, it makes the long-horizon setting faithful to real design interaction, where each successful operation changes the design state for future commands. At the same time, it exposes the central research trade-off of our work: whether long-horizon code generation should rely primarily on early execution feedback and stepwise validation, or on broader upfront planning over the full task. Our methodology therefore uses this sequential pipeline as the first concrete implementation of progressive causal-graph-guided generation for multi-step OpenROAD tasks.

K. Execution and Logging

After internal verification, committed code is executed in a live OpenROAD shell. In the single-step evaluation setting, this execution occurs once per task in a fresh process. In the sequential long-horizon setting, all accepted steps within a case execute in the same process, so that object creation and design modifications persist across sub-prompts. Output from OpenROAD is captured through a pseudo-terminal interface, cleaned to remove shell-control artifacts, and stored together with pass/fail status.

The framework also records detailed per-case results, including the generated code, verifier verdicts, OpenROAD execution outcome, output logs, controller action history, and lessons extracted during repair. In the sequential evaluation script, these results are written to an Excel workbook with step-specific columns for prompt, code, causal verdict, OpenROAD result, output, and step-level lessons. This logging structure allows later analysis of where failures occur in the long-horizon pipeline and whether they arise from graph extraction, retrieval, generation, verification, or downstream execution.

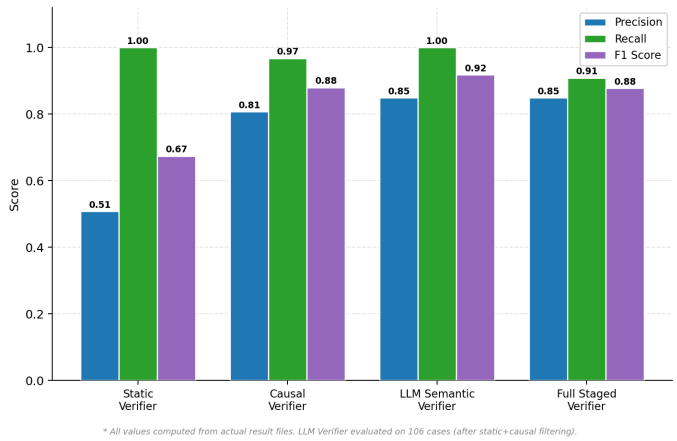

Fig. 5.

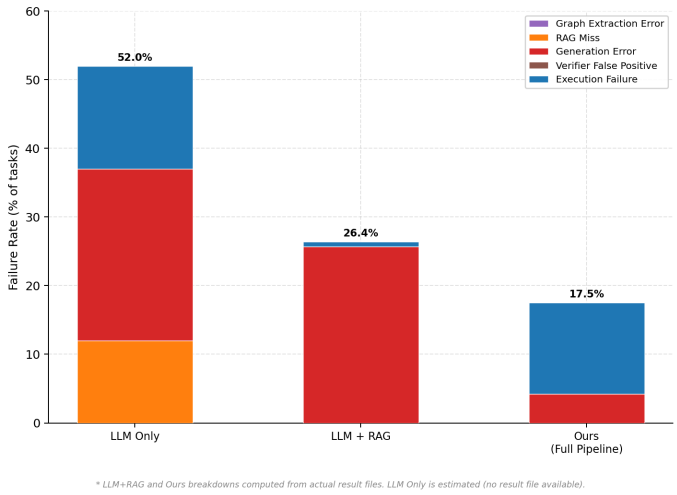

Fig. 6.

Table X: OpenROAD script generation pass rate comparison across all evaluated systems.

System Group	System	Generator	Causal Verifier	RAG	Pass Rate (%)
7B Models	7B self-correction (no verifier)	OpenROAD-Agent-7B	—	✓	16.4
	7B + GPT-4.1-mini verifier	OpenROAD-Agent-7B	GPT-4.1-mini	✓	56.4
GPT-4.1-mini	LLM only	GPT-4.1-mini	—	×	48.0
	LLM + RAG	GPT-4.1-mini	—	✓	73.0
	OpenROAD Agent	GPT-4.1-mini	—	✓	76.0
	Ours (full pipeline)	GPT-4.1-mini	GPT-4.1-mini	✓	80.0

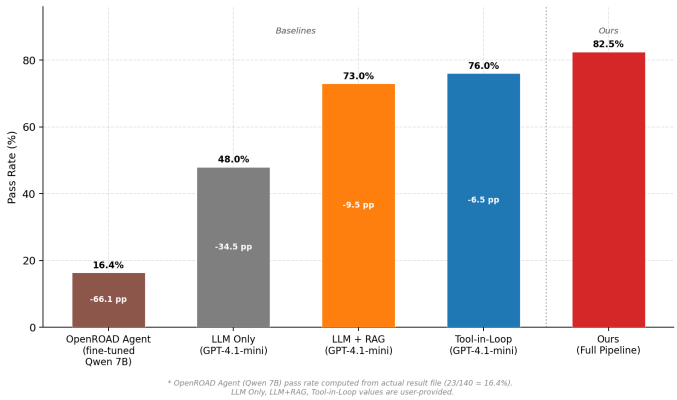

Fig. 7.